THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Chosen Software:

Git — Distributed Version Control System

Student Name	Pragya Richa Pandey
Registration Number	24BCE10797
Slot	B22
Date of Submission	31 March 2026

Table of Contents

1. Introduction

2. Part A — Origin and Philosophy

   A1. The Problem Git Was Created to Solve

   A2. The License — What It Actually Says

   A3. The Ethics of Open Source

3. Part B — Linux Footprint

4. Part C — The FOSS Ecosystem

5. Part D — Open Source vs Proprietary (Critical Analysis)

6. Shell Script Tasks

   Script 1 — System Identity Report

   Script 2 — FOSS Package Inspector

   Script 3 — Disk and Permission Auditor

   Script 4 — Log File Analyzer

   Script 5 — Open Source Manifesto Generator

7. References

1. Introduction

This report presents a structured audit of Git, the distributed version control system created by Linus Torvalds in 2005. Git is one of the most consequential open-source projects ever released — it did not merely solve a technical problem, but fundamentally changed how software is developed, shared, and maintained across the world.

The audit covers Git's origin and the problem it was built to solve, a detailed analysis of its GPL v2 license and the freedoms it grants, an ethical reflection on open-source philosophy, a practical examination of Git's footprint on a Linux system, a mapping of its broader FOSS ecosystem, and a comparison against its closest proprietary alternative. Five shell scripts are included to demonstrate practical Linux skills alongside the philosophical analysis.

Git is the invisible backbone of modern software development. Nearly every developer on the planet uses it every day — and most do not stop to ask why it exists, who made it, and what philosophy made it free. This report attempts to answer those questions properly.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem Git Was Created to Solve

To understand Git's origin, it is necessary to go back to the year 2002 and understand the state of the Linux kernel project — the most important open-source collaboration in history. By that point, the Linux kernel had thousands of contributors spread across the world, all sending in patches and modifications to Linus Torvalds. Managing all of this manually — through emailed patches and tarballs — was becoming impossible. Something had to change.

The solution the Linux kernel team adopted in 2002 was a proprietary distributed version control system called BitKeeper, developed by a company called BitMover Inc. BitMover made BitKeeper available to the Linux kernel project free of charge, under a restrictive licence that prohibited contributors from reverse-engineering the protocol or working on competing tools. This arrangement was controversial from the very beginning — many in the free software community, including Richard Stallman, argued loudly that using a proprietary tool to develop the world's most prominent free software project was philosophically inconsistent and dangerous. They were right.

The arrangement collapsed in April 2005. A kernel developer named Andrew Tridgell began reverse-engineering the BitKeeper protocol to understand how it worked. BitMover considered this a violation of the licence terms and revoked the free access it had granted to the Linux kernel project. Overnight, the kernel team had no version control system.

Linus Torvalds found himself in an impossible position. The existing open-source alternatives — CVS and Subversion (SVN) — were fundamentally unsuitable for the scale and workflow of the Linux kernel. Both were centralised systems: every operation required talking to a central server. For a project receiving thousands of patches a day from contributors worldwide, this was not just slow but structurally broken. Linus famously described CVS as an example of how not to do version control.

His requirements were clear and demanding. The system needed to be distributed — every developer should have a full copy of the entire history. It needed to be fast, capable of handling branching and merging at the scale of the kernel. It needed cryptographic integrity verification, so that every commit in history could be trusted and verified. And it needed to be free — not just free to use, but free in the fullest sense, open source, owned by no single company, impossible to have revoked.

Linus made a decision that still strikes people as remarkable: he stopped kernel development entirely for approximately ten days in April 2005 and wrote Git almost from scratch, in C. The name itself is revealing — Git is British slang for an unpleasant or contemptible person. Torvalds said, characteristically, that he names all his projects after himself: first Linux, now Git. The self-deprecating humour masked a serious intent. The first Git commit was recorded on April 7, 2005. By June 2005, barely two months later, the release of Linux kernel version 2.6.12 was managed entirely using Git.

The deeper significance of Git's origin goes beyond the technical problem it solved. Git exists because the Linux kernel project — the heart of the free software movement — had made itself dependent on proprietary infrastructure, and paid the price when that infrastructure was withdrawn. Git is, in a very real sense, a lesson about what happens when you build on a foundation you do not control. The response to that lesson was to build something better, to make it fully free and open, and to give it

away to everyone. This is the open-source philosophy expressed not as an abstract value but as a practical, urgent decision made under pressure.

Today, Git is the foundation of essentially all modern software development. GitHub, GitLab, Bitbucket, and nearly every continuous integration and deployment pipeline in existence are built on top of Git. None of it would exist without the BitKeeper incident of 2005 and one developer's decision to spend ten days solving a problem properly.

A2. The License — What It Actually Says

Git is distributed under the GNU General Public Licence version 2, commonly known as GPL v2. This is the same licence that governs the Linux kernel itself, and it is one of the most important and carefully constructed licences in the history of software. Understanding what it actually says — not just what people claim it says — is essential to understanding Git as an open-source project.

The Four Freedoms of Free Software

The philosophical foundation of the GPL comes from Richard Stallman's Free Software Foundation and its definition of the four essential freedoms that any truly free software must guarantee:

- Freedom 0 — The freedom to run the program for any purpose, without restriction.
- Freedom 1 — The freedom to study how the program works. This requires access to the source code.
- Freedom 2 — The freedom to redistribute copies of the program to anyone.
- Freedom 3 — The freedom to distribute modified versions of the program. This also requires access to the source code.

Git's GPL v2 licence grants all four of these freedoms explicitly. The source code for Git is publicly available in its entirety. Any person or organisation can download it, read it, run it, modify it, and redistribute it. There are no restrictions on purpose or use.

Can a Company Modify Git and Sell It Without Sharing Changes?

This is where the GPL v2 differs fundamentally from more permissive licences. The answer is no — with an important qualification. The GPL v2 is a copyleft licence. This means that if you distribute a modified version of Git to anyone outside your organisation, you are legally required to make the full source code of your modified version available under the same GPL v2 licence. The freedom you received must be passed on to everyone who receives your software.

This is sometimes described as a viral clause, because the licence terms propagate to any derived work that is distributed. However, this only applies to distribution. A company can take Git's source code, modify it extensively, and use their modified version entirely internally without ever being required to release those changes. It is only when they distribute the software externally that the copyleft obligation is triggered. This is why many large technology companies use GPL-licenced tools internally while keeping their proprietary applications separate.

GPL versus MIT — A Comparison

The MIT licence is the most permissive and widely used open-source licence. It places almost no restrictions on what users can do with the software. A company can take MIT-licenced code, modify it, incorporate it into a proprietary product, and sell it without ever releasing a single line of their changes. They need only include the original copyright notice.

The GPL v2 takes a different philosophical position. It prioritises the freedom of the community over the freedom of any individual user or company. By requiring that derivatives remain open, it prevents any single actor from taking a shared resource, improving it privately, and locking those improvements away.

If I were choosing a licence for something I built, the choice would depend on the goal. If the goal is maximum adoption — if I want the code to be used as widely as possible, in commercial products, in proprietary systems, everywhere — then MIT is the right choice. If the goal is to build a commons that remains a commons — to ensure that no company can build on my work and privatise the improvements — then GPL is the correct choice. For infrastructure tools like Git or the Linux kernel, where the entire point is to serve as a shared foundation, GPL makes more sense. For a library or utility that I want to see adopted universally regardless of context, MIT is more appropriate.

Free as in Free Beer versus Free as in Freedom

This distinction, popularised by Richard Stallman, is one of the most important concepts in open-source philosophy. Free beer means something that costs nothing to obtain. Free speech, or free as in freedom, means something you have the right to use, study, modify, and redistribute without restriction.

Git perfectly illustrates why the distinction matters. Git itself is free as in freedom — you have the source code, the four freedoms apply, you can fork it, modify it, run your own version. GitHub, on the other hand, is free as in free beer for many users — it costs nothing to create an account and host public repositories. But GitHub is proprietary software owned by Microsoft. You cannot inspect its source code, modify it, or run your own instance of it. You are entirely dependent on Microsoft's decisions about how it operates, what features it includes, and whether it continues to exist.

The irony is not lost on the open-source community: the most widely used interface for collaborating on open-source software is itself a closed, proprietary platform. Git the tool represents the fullest expression of software freedom. GitHub the service represents the commodification of that freedom into a commercial product. Understanding this distinction is not an academic exercise — it is directly relevant to every decision a developer makes about where to host their code and which tools to depend on.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for universal open source is compelling on its face. Transparent source code can be inspected, audited, and improved by anyone. Security vulnerabilities are more likely to be found and fixed when thousands of eyes can examine the code. Knowledge and tools are not locked behind

corporate decisions. Software that society depends upon — operating systems, communication tools, medical software — should arguably be answerable to society, not to shareholders.

But the case against is also real. Writing software is labour, and labour costs money. Many of the most critical software projects in the world suffer from chronic underfunding precisely because they are free. The OpenSSL vulnerability known as Heartbleed in 2014 affected an estimated two-thirds of all secure websites on the internet — and the project that produced it was maintained by a handful of volunteers funded by donations totalling less than one million dollars per year, supporting infrastructure worth trillions. Open source does not automatically mean well-maintained or well-funded.

There are also categories of software — medical devices, avionics, defence systems — where controlled, audited releases may be more appropriate than fully public development. And companies that invest heavily in research and development need some mechanism to recoup that investment. A world where all software is immediately open-sourced may reduce the incentive to create genuinely innovative tools in the first place.

The honest answer is that the question is too broad to answer with a single position. The appropriate licensing model depends on what the software does, who it serves, and how development is funded. The open-source philosophy is not a dogma — it is a set of values that must be applied with judgement.

Is It Ethical to Profit from Community Labor?

This is perhaps the most contested question in contemporary open-source ethics, and Git's story makes it unavoidable. Microsoft acquired GitHub in 2018 for 7.5 billion US dollars. GitHub is built entirely on Git, an open-source tool that Linus Torvalds wrote and gave away for free in 2005. Torvalds received nothing from that acquisition. The thousands of developers who contributed to Git over the decades received nothing. Microsoft received 7.5 billion dollars.

Is this ethical? The GPL v2 licence that governs Git explicitly permits this. Microsoft is not violating any licence terms. The licence was designed to keep the software free, not to ensure that contributors are compensated when commercial value is extracted from their work. The ethical question and the legal question are not the same.

The answer depends on context and behaviour. Google employs dozens of full-time engineers who contribute to the Linux kernel. Meta open-sourced PyTorch, one of the most important machine learning frameworks in existence. Red Hat built a billion-dollar business on Linux and contributed enormously to the kernel in return. These are cases where companies take from the open-source commons and give back in meaningful ways.

But many companies take freely and give back nothing. They use Linux servers, Git for version control, PostgreSQL for databases, and open-source frameworks throughout their stack, while contributing no engineering effort, no funding, and no code back to the projects they depend on. This is not illegal, but it is difficult to defend as ethical. The open-source community sustains infrastructure that generates enormous commercial value, often on the labour of volunteers, and the distribution of that value is deeply unequal.

The most defensible position is that profiting from community labour is acceptable when it is paired with genuine contribution back to that community — whether in code, funding, documentation, or infrastructure. It becomes ethically questionable when extraction is systematic and contribution is absent.

Standing on the Shoulders of Giants in Software

The phrase, attributed to Isaac Newton, describes how scientific progress builds on the accumulated knowledge of previous generations. In software, the concept is even more tangible. Every application, every website, every mobile app in existence today stands on an enormous pyramid of open-source foundations. The operating systems, the compilers, the languages, the libraries, the version control tools — almost all of it was built openly and given away freely.

Git both embodies this principle and enables it. Git itself was built by standing on decades of prior work in distributed systems, cryptographic hashing, and version control theory. And in return, Git made it possible for the modern open-source collaboration model to exist. GitHub's open-source ecosystem — where millions of developers contribute to projects they have never been paid to work on — could not function without the infrastructure Git provides.

Whether open source enables or undermines original innovation is a question worth taking seriously. The argument that it undermines innovation — that sharing code removes the financial incentive to create it — has some validity in specific contexts. But the evidence of the last thirty years points strongly in the other direction. The most innovative and transformative software ecosystems in history — Linux, Python, the web itself — were built openly. The sharing of foundational tools has consistently produced more innovation, not less, because it allows every new project to start from a higher baseline rather than rebuilding from scratch.

Part B — Linux Footprint (Unit 2)

This section documents the installation and Linux footprint of Git. All commands were executed on Ubuntu 22.04 LTS running inside a VirtualBox virtual machine.

Installation Method

Git is available in the default apt package repositories on all major Debian and Ubuntu-based Linux distributions, making installation straightforward without needing to compile from source.

```
$ sudo apt update && sudo apt install git -y
```

To verify the installation and check the installed version:

```
$ git --version git version 2.34.1
```

Git is also available via RPM-based package managers on Fedora and CentOS using dnf install git. For users who need a newer version than what the distribution provides, Git can also be compiled from source after installing dependencies such as gcc, make, and libssl-dev. For this project, the apt method was used.

Key Directories

Git's installation places files across several standard Linux filesystem locations according to the Filesystem Hierarchy Standard (FHS):

- /usr/bin/git — The main Git executable binary.
- /usr/lib/git-core/ — Supporting Git subcommand executables (git-commit, git-push, etc.).
- /usr/share/git-core/ — Shared data files, templates for new repositories.
- /usr/share/doc/git/ — Documentation files including the changelog and copyright notice.
- /usr/share/man/ — Manual pages accessible via man git and its subcommands.
- ~/.gitconfig — Per-user configuration file storing name, email, preferences.
- .git/ — The hidden directory created inside every repository by git init, storing all version history, objects, refs, and configuration.

The most important directory in day-to-day use is the .git/ directory within each repository. This directory contains the entire version history as a content-addressable object store, and it is what makes Git's distributed model work — every clone is a complete repository.

User and Group

Git runs entirely as the current logged-in user. Unlike a daemon or service like Apache or MySQL, Git does not create a dedicated system user or group. When you run git commit, it executes as you, with your file permissions.

```
$ ls -la /usr/bin/git -rwxr-xr-x 1 root root 3358024 git
```

The Git binary itself is owned by root and readable by all users. This is correct and expected behaviour — the binary must be executable by any user, but only the system administrator should be able to replace or modify it. This matters for security because it prevents an unprivileged user from replacing the Git binary with malicious code.

Service Management

Git itself is not a persistent service and does not run as a daemon under systemd in a standard installation. It is a command-line tool that runs, completes, and exits. However, for hosting Git repositories over a network, a Git daemon can be run:

```
$ sudo systemctl start git-daemon@.service $ sudo systemctl enable git-daemon@.service $ sudo systemctl status git-daemon@.service $ sudo systemctl stop git-daemon@.service
```

In practice, most teams use SSH or HTTPS to host Git repositories rather than the native Git daemon protocol, as these provide authentication and encryption.

Update Model

Security patches and updates to Git are distributed through the standard Linux distribution package management system. When a vulnerability is discovered in Git, the Ubuntu and Debian security teams backport the patch and push it to the security update repository. Users receive the update through:

```
$ sudo apt update && sudo apt upgrade
```

This model means that security patches to Git flow through the distribution's security infrastructure rather than requiring users to manually download and compile new versions. The open-source nature of Git is essential to this process — the security teams can read the source, understand the vulnerability, and prepare a patch, all of which would be impossible with closed-source software.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies

Git is written primarily in C, with some components in shell script and Perl. Its key dependencies on a Linux system include:

- OpenSSL or GnuTLS — for cryptographic operations including HTTPS transport and object hashing.
- libcurl — for HTTP and HTTPS communication with remote repositories.
- zlib — for compression and decompression of repository objects.
- Perl — used for some ancillary Git utilities and the git-svn bridge.
- POSIX shell — many Git helper scripts rely on a standard POSIX-compliant shell environment.
- GCC or Clang — required only for building from source, not for using the installed binary.

One of Git's design strengths is intentional minimalism in its dependencies. The core Git functionality — creating commits, branching, merging, diffing — depends only on zlib and standard POSIX interfaces. This is why Git can be compiled and run on an enormous variety of platforms.

What Git Has Enabled and Inspired

Git's impact on the software ecosystem is extraordinarily broad. It did not merely replace BitKeeper — it created the infrastructure for a new model of distributed, global software collaboration.

- GitHub (2008) — Built on Git, GitHub created the pull request workflow that now defines how open-source collaboration works. Acquired by Microsoft in 2018 for 7.5 billion dollars.
- GitLab (2011) — An open-source alternative to GitHub, providing full DevOps capabilities. Itself a large open-source project.
- Bitbucket — Atlassian's Git hosting platform, widely used in enterprise environments.
- Gitea and Forgejo — Lightweight, self-hostable Git servers for teams who want full control over their infrastructure.
- CI/CD pipelines — GitHub Actions, GitLab CI, Jenkins, CircleCI, and virtually every continuous integration system in existence is built around Git hooks and events.
- Git-LFS (Large File Storage) — An extension for handling large binary files in Git repositories.
- Mercurial — A contemporary of Git also created in response to the BitKeeper incident, now largely superseded by Git but still in use at some organisations.

Git and the LAMP Stack / Web Relationship

Git is not itself part of the traditional LAMP stack (Linux, Apache, MySQL, PHP), but it is the tool that deploys and manages every component of the LAMP stack and the applications built on it. Every web application, every configuration file, every Apache virtual host definition, every PHP codebase is managed using Git in any modern development workflow. Git connects to the web ecosystem through:

- Deployment pipelines — git push triggers automated deployment to web servers.

- Configuration management — Infrastructure-as-code tools like Ansible and Terraform store their configurations in Git repositories.
- Package registries — npm, PyPI, and RubyGems all use Git repositories as the authoritative source for package code.

Community and Governance

Git's development community operates through a mailing list model rather than a web-based issue tracker or pull request system. The main development list is git@vger.kernel.org, where patches are submitted as emails and reviewed publicly. This is the same model used by the Linux kernel itself.

Governance of Git is relatively informal compared to larger foundations. Junio C Hamano has served as Git's primary maintainer since Linus Torvalds handed off the project in July 2005 — just three months after he created it. Hamano has maintained the project continuously for nearly two decades, reviewing and integrating contributions from hundreds of developers worldwide. There is no formal board or committee — the project is governed by meritocracy, consensus on the mailing list, and the maintainer's judgement.

The Software Freedom Conservancy holds the trademarks and provides organisational support for Git as a project, but does not control its technical direction. Major contributors include employees of Google, Microsoft, GitHub, and other technology companies, as well as a large number of independent volunteers.

Part D — Open Source vs Proprietary (Critical Analysis)

Git's closest proprietary alternative in the enterprise version control space is Perforce Helix Core, commonly known simply as Perforce. It is widely used in game development, semiconductor design, and large enterprise software projects. A comparison of the two systems across key dimensions follows.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free — no licensing cost. Hosting costs only if using paid platforms like GitHub Enterprise.	Expensive per-user licences. Enterprise deployments can cost hundreds of thousands of dollars annually.
Security (code auditability)	Fully auditable — anyone can read the source code and verify its behaviour. Vulnerabilities are found and fixed publicly.	Closed source — security depends entirely on Perforce's internal testing and disclosure policies. No independent audit is possible.
Support and reliability	Community support is vast and high quality. Commercial support available from many vendors. GitHub offers SLAs for Enterprise.	Professional commercial support with SLAs. Single vendor dependency creates risk if company changes direction.
Freedom to modify	GPL v2 — full source available. You can fork, modify, and deploy your own version of Git.	No modification rights. All changes to the tool itself are controlled by Perforce Inc.
Community vs corporate control	Community-governed. No single company controls Git's development direction. Linus Torvalds and contributors worldwide own its future.	Entirely controlled by Perforce Inc. Product roadmap is driven by commercial priorities.
Scalability	Excellent for most projects. Can struggle with very large binary files and monorepos at extreme scale (addressed by Git-LFS and sparse checkout).	Designed specifically for very large binary repositories and monorepos at massive scale. Preferred in game development for this reason.
Overall verdict	The right choice for the overwhelming majority of software projects.	Justified only in specific, extreme-scale scenarios with large binary assets.

Conclusion and Verdict

For the overwhelming majority of software development contexts, Git is unambiguously the better choice. It is free, fully auditable, governed by the community rather than a single corporation, and supported by the largest and most active developer ecosystem in history. The learning curve is real — Git's command-line interface is complex and its mental model takes time to internalise — but the investment is entirely worthwhile. Every developer in the world already uses Git. Every tool, every

platform, every workflow assumes it. Choosing a proprietary alternative means working against the grain of the entire industry.

Perforce Helix Core is not a bad product — it solves a specific problem (extremely large binary repositories at enterprise scale) very well. But for standard software development, it represents an unnecessary cost, a vendor dependency, and a philosophical compromise that has no practical justification. I would choose Git for any project I worked on, and I would contribute to it. The project's governance model — a publicly archived mailing list, a single long-serving maintainer, community-driven development — is not glamorous, but it has produced one of the most reliable and widely adopted tools in the history of computing. That is not an accident. It is the result of building in the open, governing by merit, and making the software free in the fullest sense of the word.

Shell Script Tasks

All five scripts are written for Linux (Bash), include explanatory comments throughout, and must be run on a real Linux system. Each script is documented below with its source code and an explanation of the shell scripting concepts used.

Script 1 — System Identity Report

This script displays a welcome screen introducing the Linux system, showing key system information including the distribution, kernel version, logged-in user, home directory, uptime, and the open-source licence covering the OS.

Shell concepts used: variables, echo with formatting, command substitution with $(), reading from /etc/os-release, and basic string formatting.

```
#!/bin/bash
```

```
#!/bin/bash
# Script 1: System Identity Report
# Author: [Your Name] | Reg: [Your Reg No] | Course: Open Source Software
# Purpose: Display a system welcome screen with OS and environment details

# --- Variables ---
STUDENT_NAME="[Your Name]"         # Student name for display
SOFTWARE_CHOICE="Git"              # Chosen open-source software

# --- Gather system information using command substitution ---
KERNEL=$(uname -r)                 # Get current kernel version
USER_NAME=$(whoami)                # Get the current logged-in username
HOME_DIR=$HOME                     # Home directory from environment variable
UPTIME=$(uptime -p)                # Human-readable uptime
CURRENT_DATE=$(date '+%d %B %Y %H:%M:%S')  # Formatted date and time

# --- Read distro name from /etc/os-release (standard on all modern distros) ---
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d'=' -f2 | tr -d '"')

# --- Display the report ---
echo "==================================================="
echo "  Open Source Audit — System Identity Report"
echo "  Student : $STUDENT_NAME"
echo "  Software: $SOFTWARE_CHOICE"
echo "==================================================="
echo ""
echo "  Distribution : $DISTRO"
echo "  Kernel Version: $KERNEL"
echo "  Logged-in User: $USER_NAME"
echo "  Home Directory: $HOME_DIR"
echo "  System Uptime : $UPTIME"
echo "  Date and Time : $CURRENT_DATE"
echo ""
echo "  OS Licence    : The Linux kernel is licenced under GPL v2."
echo "  This means you have the freedom to run, study, modify,"
echo "  and redistribute this operating system and its source code."
echo "==================================================="
```

[Insert screenshot of script running here]

Script 2 — FOSS Package Inspector

This script checks whether Git is installed on the system, retrieves its version and licence information using the package manager, and uses a case statement to print a philosophy note about the selected package.

Shell concepts used: if-then-else for package detection, dpkg -l and grep for version information, case statement for branching on package name, pipe operator for filtering output.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: [Your Name] | Course: Open Source Software
# Purpose: Check if a FOSS package is installed and display its details

PACKAGE="git"   # The package to inspect — change as needed

# --- Check if the package is installed using dpkg ---
if dpkg -l $PACKAGE 2>/dev/null | grep -q '^ii'; then
    echo "Package '$PACKAGE' is installed on this system."
    echo ""
    # Display version, description and licence info
    dpkg -l $PACKAGE | grep '^ii' | awk '{print "  Version : " $3}'
    dpkg -s $PACKAGE | grep -E 'Description:|Homepage:'
else
    echo "Package '$PACKAGE' is NOT installed."
    echo "Install it with: sudo apt install $PACKAGE"
fi

echo ""
# --- Case statement: print a philosophy note based on the package name ---
case $PACKAGE in
    git)
        echo "Git: born from the BitKeeper incident — proof that freedom cannot be
borrowed."
        ;;
    httpd|apache2)
        echo "Apache: the web server that built the open internet — free since 1995."
        ;;
    mysql|mariadb)
        echo "MySQL: open source at the heart of millions of applications worldwide."
        ;;
    vlc)
        echo "VLC: started by students in Paris — plays everything, costs nothing."
        ;;
    python3)
        echo "Python: shaped entirely by community since 1991 — language of science
and learning."
        ;;
    *)
        echo "Open source: built by many, owned by none, available to all."
        ;;
esac
```

[Insert screenshot of script running here]

Script 3 — Disk and Permission Auditor

This script loops through a predefined list of important system directories and reports the permissions, owner, group, and disk usage of each. It also checks specifically for Git's configuration directory.

Shell concepts used: for loop iterating over an array, conditional directory check with [-d], du for disk usage, ls -ld with awk to extract permission fields, and compound commands.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: [Your Name] | Course: Open Source Software
# Purpose: Audit permissions and disk usage of key system directories

# Array of important system directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share/git-core")

echo "========================================"
echo "  Directory Audit Report"
echo "========================================"
echo ""

# Loop through each directory in the array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, group using awk on ls output
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        # Get human-readable size; suppress permission errors with 2>/dev/null
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "  Directory : $DIR"
        echo "  Permissions: $PERMS  Owner: $OWNER  Group: $GROUP"
        echo "  Disk Usage : ${SIZE:-unknown}"
        echo "  ----------------------------------------"
    else
        echo "  $DIR does not exist on this system."
        echo "  ----------------------------------------"
    fi
done

echo ""
# --- Check specifically for Git user configuration file ---
echo "  Checking for Git user config file (~/.gitconfig)..."
if [ -f "$HOME/.gitconfig" ]; then
    echo "  Found: $HOME/.gitconfig"
    ls -la "$HOME/.gitconfig"
else
    echo "  Not found. Run 'git config --global user.name' to create it."
fi
```

[Insert screenshot of script running here]

Script 4 — Log File Analyzer

This script reads a log file line by line, counts how many lines contain a specified keyword, prints the count, and outputs the last five matching lines. The log file path and keyword are passed as command-line arguments.

Shell concepts used: command-line arguments with $1 and $2, default values with ${var:-default}, file existence check with [! -f], while read loop for line-by-line processing, if-then inside the loop, counter variable with arithmetic expansion, and tail with grep for final output.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: [Your Name] | Course: Open Source Software
# Usage: ./log_analyzer.sh /var/log/syslog ERROR
# Purpose: Count and display lines matching a keyword in a log file

# $1 is the log file path, $2 is the keyword (defaults to 'error' if not given)
LOGFILE=$1
KEYWORD=${2:-"error"}    # Use second argument or default to "error"
COUNT=0                   # Initialise match counter

# --- Validate that a log file argument was provided ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# --- Check that the file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."
    exit 1
fi

# --- Check that the file is not empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: File '$LOGFILE' is empty. No lines to analyse."
    exit 0
fi

echo "Analysing: $LOGFILE"
echo "Searching for keyword: '$KEYWORD' (case-insensitive)"
echo "-------------------------------------------"

# --- Read file line by line using while-read loop ---
while IFS= read -r LINE; do
    # Check if this line contains the keyword (case-insensitive with -i flag)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter using arithmetic expansion
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""

# --- Print the last 5 matching lines for context ---
```

```
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    echo "-----------------------------------------"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

[Insert screenshot of script running here]

Script 5 — The Open Source Manifesto Generator

This script interactively asks the user three questions about their relationship with open-source software, then composes a personalised philosophy statement from their answers and saves it to a text file.

Shell concepts used: read for interactive user input, string concatenation to build the manifesto paragraph, writing output to a file with > and >>, the date command, and a comment demonstrating the alias concept.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: [Your Name] | Course: Open Source Software
# Purpose: Generate a personalised open source philosophy statement

# Note on aliases: In a shell session you could define:
#   alias manifesto='./script5.sh'
# This would let you type 'manifesto' instead of './script5.sh'
# Aliases make long commands shorter — a core shell productivity concept.

echo "=========================================="
echo "   Open Source Manifesto Generator"
echo "=========================================="
echo "Answer three questions to generate your"
echo "personal open source philosophy statement."
echo ""

# --- Gather user input with read ---
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

# --- Get current date and generate output filename ---
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# --- Compose the manifesto by concatenating strings and writing to file ---
echo "Open Source Manifesto" > "$OUTPUT"
echo "Generated on: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Build the manifesto paragraph using the three answers
echo "Every day, I rely on $TOOL — a tool that someone built and chose" >> "$OUTPUT"
echo "to share freely with the world. That act of sharing is what freedom" >>
"$OUTPUT"
```

```
echo "means to me: $FREEDOM. It is not just about the absence of cost, but" >>
"$OUTPUT"
echo "the presence of trust — the belief that knowledge and tools belong to" >>
"$OUTPUT"
echo "everyone, not just to those who can afford them. In that same spirit," >>
"$OUTPUT"
echo "if I were to build one thing and give it away, it would be $BUILD." >>
"$OUTPUT"
echo "Not because I have to, but because that is what standing on the" >> "$OUTPUT"
echo "shoulders of giants looks like when you are the one building next." >>
"$OUTPUT"

echo ""
echo "============================================"
echo "Manifesto saved to: $OUTPUT"
echo "============================================"
echo ""
cat "$OUTPUT"
```

[Insert screenshot of script running here]

7. References

The following sources were used as references for this report. All written analysis and conclusions are original work.

- GNU Project — The Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative — The Open Source Definition: opensource.org/osd
- Eric S. Raymond — The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — Just for Fun (autobiography, 2001)
- Git official website and documentation: git-scm.com
- GPL v2 licence text: gnu.org/licenses/old-licenses/gpl-2.0.html
- SPDX Licence List: spdx.org/licenses
- The Linux Command Line by William Shotts: linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Junio C Hamano's Git maintainership history: public-inbox.org/git
- BitKeeper and the Linux Kernel — documented history on lwn.net
- Software Freedom Conservancy — Git project page: sfconservancy.org